Data Mining Notes

Chapter 1: Introduction

- Data mining is a technology that blends traditional data analysis methods with sophisticated algorithms for processing large volumes of data. It has also opened up exciting opportunities for exploring and analyzing new types of data and for analyzing old types of data in new ways.
- **Business** Point-of-sale data collection (bar code scanners, radio frequency identification (RFID), and smart card technology) have allowed retailers to collect up-to-the-minute data about customer purchases at the checkout counters of their stores.
- Data mining techniques can be used to support a wide range of business intelligence applications such as **customer profiling, targeted marketing, workflow management, store layout, and fraud detection**. It can also help retailers answer important business questions such as "**Who are the most profitable customers?**" "**What products can be cross-sold or up-sold?**" and "**What is the revenue outlook of the company for next year?"**
- **Medicine, Science, and Engineering** Researchers in medicine, science, and engineering are rapidly accumulating data that is key to important new discoveries.

  Techniques developed in data mining can aid Earth scientists in answering questions such as "**What is the relationship between the frequency and intensity of ecosystem disturbances such as droughts and hurricanes to global warming?**" "**How is land surface precipitation and temperature affected by ocean surface temperature?**" and "**How well can we predict the beginning and end of the growing season for a region?**"

What is Data Mining ?

- **Data mining** is the process of automatically discovering useful information in large data repositories. Data mining techniques are deployed to scour large databases in order to find novel and useful patterns that might otherwise remain unknown.
- For example, looking up individual records using a database management system or finding particular Web pages via a query to an Internet search engine are tasks related to the area of **information retrieval**.

DEEPAK RAJPUT

Data Mining and Knowledge Discovery

- Data mining is an integral part of knowledge discovery in databases (KDD), which is the overall process of converting raw data into useful information, as shown in Figure 1.1. This process consists of a series of transformation steps, from data preprocessing to postprocessing of data mining results.

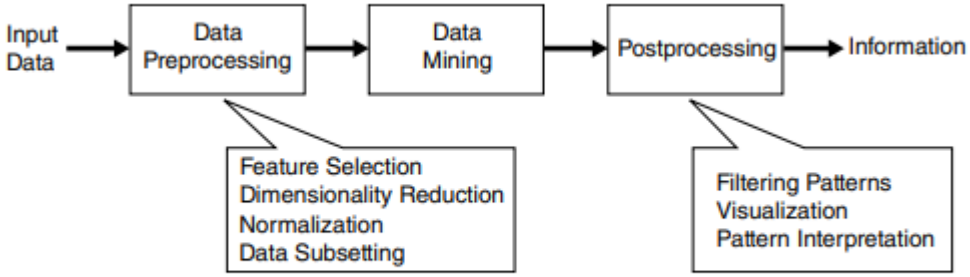


Figure 1.1. The process of knowledge discovery in databases (KDD).

- The **input data** can be stored in a variety of formats (flat files, spread-sheets, or relational tables) and may reside in a centralized data repository or be distributed across multiple sites.
- The purpose of **preprocessing** is to transform the raw input data into an appropriate format for subsequent analysis. The steps involved in data **preprocessing** include fusing data from multiple sources, cleaning data to remove noise and duplicate observations, and selecting records and features that are relevant to the data mining task at hand.
- "**Closing the loop**" is the phrase often used to refer to the process of integrating data mining results into decision support systems.
- In business applications, the insights offered by data mining results can be integrated with campaign management tools so that effective marketing promotions can be conducted and tested. Such integration requires a **postprocessing** step that ensures that only valid and useful results are incorporated into the decision support system.
- Statistical measures or hypothesis testing methods can also be applied during postprocessing to eliminate spurious data mining results.

Motivating Challenge

- As mentioned earlier, **traditional data analysis** techniques have often encountered practical difficulties in meeting the challenges posed by new data sets. The following are some of the specific **challenges** that motivated the development of data mining.

DEEPAK RAJ

Scalability

- Because of advances in data generation and collection, data sets with sizes of gigabytes, terabytes, or even petabytes are becoming common. If data mining algorithms are to handle these massive data sets, then they must be scalable.
- Many data mining algorithms employ special search strategies to handle exponential search problems.
- Scalability may also require the implementation of novel data structures to access individual records in an efficient manner. Scalability can also be improved by using sampling or developing parallel and distributed algorithms.

High Dimensionality

- It is now common to encounter data sets with hundreds or thousands of attributes instead of the handful common a few decades ago.
- In bioinformatics, progress in microarray technology has produced gene expression data involving thousands of features.
- Data sets with temporal or spatial components also tend to have high dimensionality.
- For example, consider a data set that contains measurements of temperature at various locations. If the temperature measurements are taken repeatedly for an extended period, the number of dimensions (features) increases in proportion to Motivating Challenges the number of measurements taken.
- Traditional data analysis techniques that were developed for low-dimensional data often do not work well for such high-dimensional data. Also, for some data analysis algorithms, the computational complexity increases rapidly as the dimensionality (the number of features) increases.

Heterogeneous and Complex Data

- Traditional data analysis methods often deal with data sets containing attributes of the same type, either continuous or categorical.
- Examples of such non-traditional types of data include collections of Web pages containing semi-structured text and hyperlinks; DNA data with sequential and three-dimensional structure; and climate data that consists of time series measurements (temperature, pressure, etc.) at various locations on the Earth's surface.

DEEPAK RAJPUT

- Techniques developed for mining such complex objects should take into consideration relationships in the data, such as temporal and spatial autocorrelation, graph connectivity, and parent-child relationships between the elements in semi-structured text and XML documents.

Data Ownership and Distribution

- Sometimes, the data needed for an analysis is not stored in one location or owned by one organization. Instead, the data is geographically distributed among resources belonging to multiple entities. This requires the development of distributed data mining techniques. Among the key challenges faced by distributed data mining algorithms include
  (1) How to reduce the amount of communication needed to perform the distributed computation,
  (2) How to effectively consolidate the data mining results obtained from multiple sources, and
  (3) How to address data security issues.

Non-traditional Analysis

- The traditional statistical approach is based on a hypothesize-and-test paradigm. In other words, a hypothesis is proposed, an experiment is designed to gather the data, and then the data is analyzed with respect to the hypothesis. Unfortunately, this process is extremely labor-intensive.
- Current data analysis tasks often require the generation and evaluation of thousands of hypotheses, and consequently, the development of some data mining techniques has been motivated by the desire to automate the process of hypothesis generation and evaluation.

The Origins of Data Mining

- In particular, data mining draws upon ideas, such as
  (1) Sampling, estimation, and hypothesis testing from statistics and
  (2) search algorithms, modeling techniques, and learning theories from artificial intelligence, pattern recognition, and machine learning.
- Data mining has also been quick to adopt ideas from other areas, including optimization, evolutionary computing, information theory, signal processing, visualization, and information retrieval.

DEEPAK RAJPUT

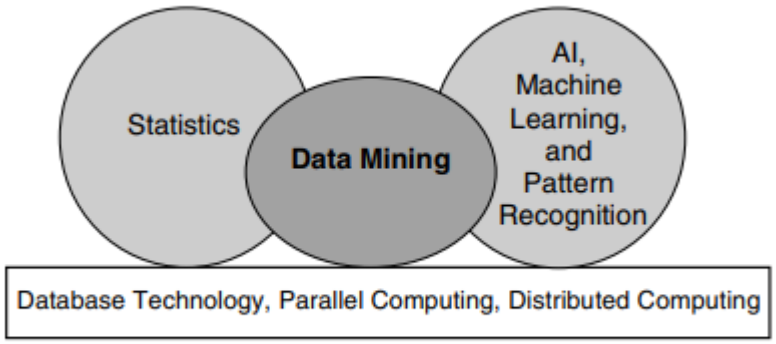


Figure 1.2. Data mining as a confluence of many disciplines.

- A number of other areas also play key supporting roles. In particular, database systems are needed to provide support for efficient storage, indexing, and query processing. Techniques from high performance (parallel) computing are often important in addressing the massive size of some data sets. Distributed techniques can also help address the issue of size and are essential when the data cannot be gathered in one location.

Data Mining Tasks

- Data mining tasks are generally divided into two major categories:

Predictive tasks

- The objective of these tasks is to predict the value of a particular attribute based on the values of other attributes. The attribute to be predicted is commonly known as the **target or dependent variable**, while the attributes used for making the prediction are known as the **explanatory** or **independent variables**.

Descriptive tasks

- The objective is to **derive patterns (correlations, trends, clusters, trajectories, and anomalies)** that summarize the underlying relationships in data. Descriptive data mining tasks are often exploratory in nature and frequently require postprocessing techniques to validate and explain the results.

DEEPAK RAJPUT

Figure 1.3 illustrates four of the core data mining tasks that are described in the remainder of this book.

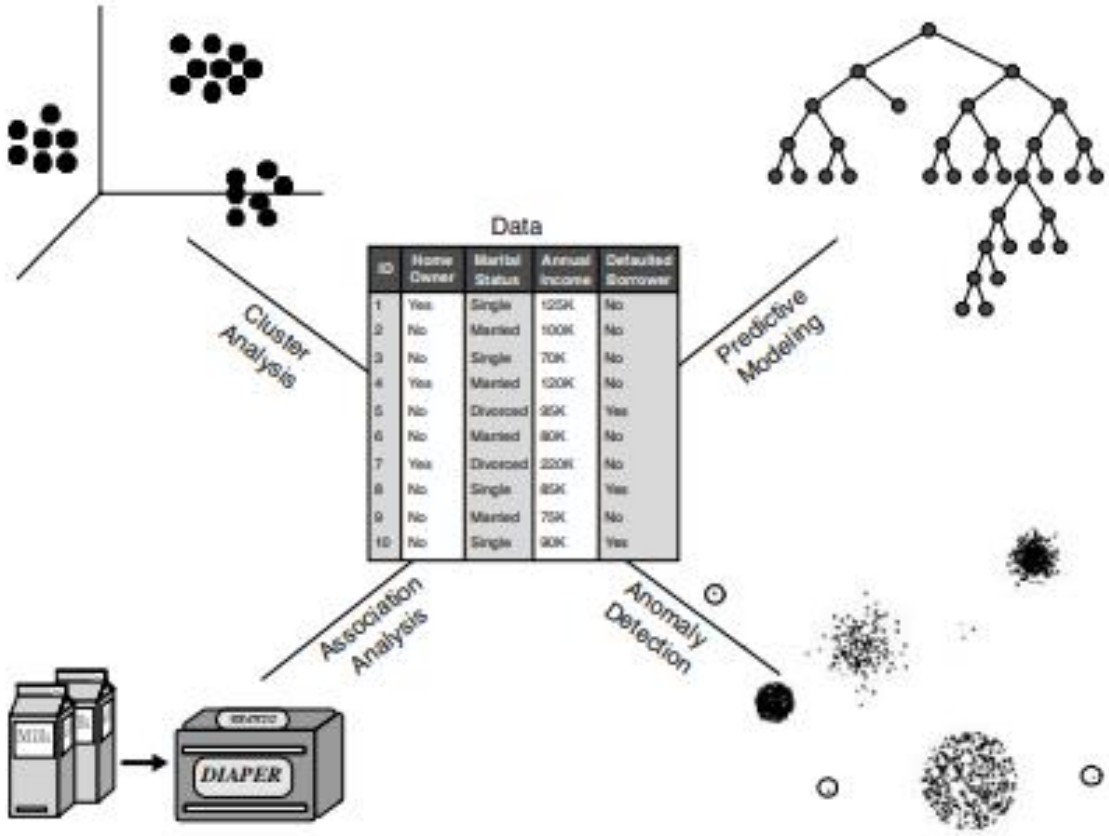


Figure 1.3. Four of the core data mining tasks.

Predictive modeling

- **Predictive** refers to the task of building a model for the target variable as a function of the explanatory variables. There are two types of predictive modeling tasks:
  - **classification**, which is used for discrete target variables, and
  - **regression**, which is used for continuous target variables.

Example 1.1 (Predicting the Type of a Flower)

- Consider the task of predicting a species of flower based on the characteristics of the flower. In particular, consider classifying an Iris flower as to whether it belongs to one of the following three Iris species: Setosa, Versicolour, or Virginica. To perform this task, we need a data set containing the characteristics of various flowers of these three species. A data set with this type of information is the

well-known Iris data set from the UCI Machine Learning Repository at http://www.ics.uci.edu/~mlearn.

- In addition to the species of a flower, this data set contains four other attributes: sepal width, sepal length, petal length, and petal width. (The Iris data set and its attributes are described further in Section 3.1.) Figure 1.4 shows a plot of petal width versus petal length for the 150 flowers in the Iris data set. Petal width is broken into the categories low, medium, and high, which correspond to the intervals [0, 0.75), [0.75, 1.75), [1.75, ∞), respectively. Also, petal length is broken into categories low, medium, and high, which correspond to the intervals [0, 2.5), [2.5, 5), [5, ∞), respectively. Based on these categories of petal width and length, the following rules can be derived:

  Petal width low and petal length low implies Setosa.
  Petal width medium and petal length medium implies Versicolour.
  Petal width high and petal length high implies Virginica.

  While these rules do not classify all the flowers, they do a good (but not perfect) job of classifying most of the flowers. Note that flowers from the Setosa species are well separated from the Versicolour and Virginica species with respect to petal width and length, but the latter two species overlap somewhat with respect to these attributes.

DEEPAK RAJPUT

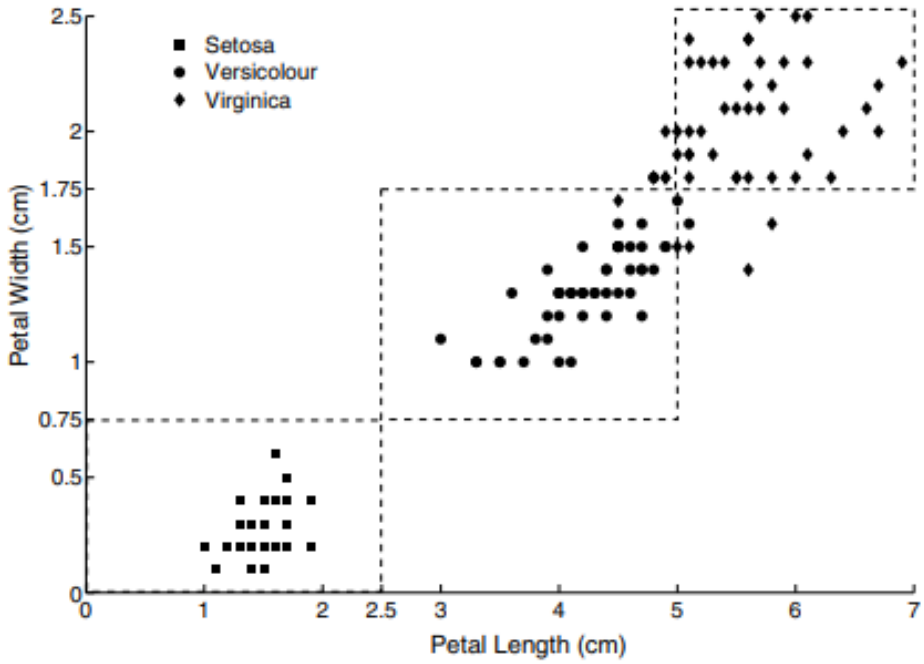


Figure 1.4. Petal width versus petal length for 150 Iris flowers.

hhh

Association analysis

- **Association analysis** is used to discover patterns that describe strongly associated features in the data. The discovered patterns are typically represented in the form of implication rules or feature subsets. Because of the exponential size of its search space, the goal of association analysis is to extract the most interesting patterns in an efficient manner. Useful applications of association analysis include finding groups of genes that have related functionality, identifying Web pages that are accessed together, or understanding the relationships between different elements of Earth's climate system.

Example 1.2 (Market Basket Analysis)

- The transactions shown in Table 1.1 illustrate point-of-sale data collected at the checkout counters of a grocery store. Association analysis can be applied to find items that are frequently bought together by customers. For example, we may discover the rule {**Diapers**} → {**Milk**}, which suggests that customers who buy diapers also tend to buy milk. This type of rule can be used to identify potential cross-selling opportunities among related items.

DEEPAK RA

Table 1.1. Market basket data.

Transaction ID	Items
1	{Bread, Butter, Diapers, Milk}
2	{Coffee, Sugar, Cookies, Salmon}
3	{Bread, Butter, Coffee, Diapers, Milk, Eggs}
4	{Bread, Butter, Salmon, Chicken}
5	{Eggs, Bread, Butter}
6	{Salmon, Diapers, Milk}
7	{Bread, Tea, Sugar, Eggs}
8	{Coffee, Sugar, Chicken, Eggs}
9	{Bread, Diapers, Milk, Salt}
10	{Tea, Eggs, Cookies, Diapers, Milk}

Cluster analysis

- **Cluster analysis** seeks to find groups of closely related observations so that observations that belong to the same cluster are more similar to each other than observations that belong to other clusters. Clustering has been used to group sets of related customers, find areas of the ocean that have a significant impact on the Earth's climate, and compress data.

Example 1.3 (Document Clustering)

- The collection of news articles shown in Table 1.2 can be grouped based on their respective topics. Each article is represented as a set of word-frequency pairs (w, c), where w is a word and c is the number of times the word appears in the article. There are two natural clusters in the data set. The first cluster consists of the first four articles, which correspond to news about the economy, while the second cluster contains the last four articles, which correspond to news about health care. A good clustering algorithm should be able to identify these two clusters based on the similarity between words that appear in the articles.

DEEPAK RAJPU

Table 1.2. Collection of news articles.

Article	Words
1	dollar: 1, industry: 4, country: 2, loan: 3, deal: 2, government: 2
2	machinery: 2, labor: 3, market: 4, industry: 2, work: 3, country: 1
3	job: 5, inflation: 3, rise: 2, jobless: 2, market: 3, country: 2, index: 3
4	domestic: 3, forecast: 2, gain: 1, market: 2, sale: 3, price: 2
5	patient: 4, symptom: 2, drug: 3, health: 2, clinic: 2, doctor: 2
6	pharmaceutical: 2, company: 3, drug: 2, vaccine: 1, flu: 3
7	death: 2, cancer: 4, drug: 3, public: 4, health: 3, director: 2
8	medical: 2, cost: 3, increase: 2, patient: 2, health: 3, care: 1

Anomaly detection

- **Anomaly detection** is the task of identifying observations whose characteristics are significantly different from the rest of the data. Such observations are known as **anomalies** or **outliers**. The goal of an anomaly detection agorithm is to discover the real anomalies and avoid falsely labeling normal objects as anomalous. In other words, a good anomaly detector must have a high detection rate and a low false alarm rate. Applications of anomaly detection include the detection of fraud, network intrusions, unusual patterns of disease, and ecosystem disturbances.

Example 1.4 (Credit Card Fraud Detection)

- A credit card company records the transactions made by every credit card holder, along with personal information such as credit limit, age, annual income, and address. Since the number of fraudulent cases is relatively small compared to the number of legitimate transactions, anomaly detection techniques can be applied to build a profile of legitimate transactions for the users. When a new transaction arrives, it is compared against the profile of the user. If the characteristics of the transaction are very different from the previously created profile, then the transaction is flagged as potentially fraudulent.

DEEPAK RAJPUT

Data Mining Notes

Chapter 2: Data

The Type of Data

- Data sets differ in a number of ways. For example, the attributes used to describe data objects can be of different types—**quantitative** or **qualitative**—and data sets may have special characteristics; e.g., some data sets contain time series or objects with explicit relationships to one another. Not surprisingly, the type of data determines which tools and techniques can be used to analyze the data. Furthermore, new research in data mining is often driven by the need to accommodate new application areas and their new types of data.

The Quality of the Data

- Data is often far from perfect. While most data mining techniques can tolerate some level of imperfection in the data, a focus on understanding and improving data quality typically improves the quality of the resulting analysis. **Data quality issues** that often need to be addressed include the **presence of noise and outliers; missing, inconsistent, or duplicate data; and data that is biased** or, in some other way, unrepresentative of the phenomenon or population that the data is supposed to describe.

Pre-processing Steps to Make the Data More Suitable for Data Mining

- Often, the raw data must be processed in order to make it suitable for analysis. While one objective may be to improve data quality, other goals focus on modifying the data so that it better fits a specified **data mining technique or tool**. For example, a continuous attribute, e.g., length, may need to be transformed into an attribute with discrete categories, e.g., short, medium, or long, in order to apply a particular technique.

Analyzing Data in Terms of Its Relationships

- One approach to data analysis is to find **relationships** among the data objects and then perform the remaining analysis using these relationships rather than the data objects themselves. For instance, we can compute the similarity or distance between pairs of objects and then perform the analysis—**clustering, classification, or anomaly detection** based on these similarities or distances. There are many such similarity or distance

DEEPAK RAJPUT

measures, and the proper choice depends on the type of data and the particular application.

Types of Data

A **data** set can often be viewed as a combined collection of **data objects**. Other names for a data object are **record**, **point**, **vector**, **pattern**, **event**, **case**, **sample**, **observation**, or **entity**.

In turn, data objects are described by a number of attributes that capture the basic characteristics of an object, such as the mass of a physical object or the time at which an event occurred. Other names for an attribute are **variable**, **characteristic**, **field**, **feature**, or **dimension.**

Attributes and Measurement

Attribute :- An **attribute** is a property or characteristic of an object that may vary, either from one object to another or from one time to another.

> **For example**, eye color varies from person to person, while the temperature of an object varies over time. Note that eye color is a symbolic attribute with a small number of possible values {brown, black, blue, green, hazel, etc.}, while temperature is a numerical attribute with a potentially unlimited number of values.

Measurement :- The process of **measurement** is the application of a measurement scale to associate a value with a particular attribute of a specific object.

Measurement Scale :- A measurement scale is a rule (function) that associates a numerical or symbolic value with an attribute of an object.

The Different Types of Attributes

The following properties (operations) of numbers are typically used to describe attributes.

1. Distinctness; = and !=

2. Order; <, ≤, >, and ≥

3. Addition; + and −

4. Multiplication; ∗ and /

DEEPAK RAJPUT

Table 2.2. Different attribute types.

Attribute Type		Description	Examples	Operations
Categorical (Qualitative)	Nominal	The values of a nominal attribute are just different names; i.e., nominal values provide only enough information to distinguish one object from another. $(=, \neq)$	zip codes, employee ID numbers, eye color, gender	mode, entropy, contingency correlation, χ^2 test
	Ordinal	The values of an ordinal attribute provide enough information to order objects. $(<, >)$	hardness of minerals, $\{good, better, best\}$, grades, street numbers	median, percentiles, rank correlation, run tests, sign tests
Numeric (Quantitative)	Interval	For interval attributes, the differences between values are meaningful, i.e., a unit of measurement exists. $(+, -)$	calendar dates, temperature in Celsius or Fahrenheit	mean, standard deviatio Pearson's correlation, t and F tests
	Ratio	For ratio variables, both differences and ratios are meaningful. $(*, /)$	temperature in Kelvin, monetary quantities, counts, age, mass, length, electrical current	geometric mean, harmonic mean, percent variation

Table 2.3. Transformations that define attribute levels.

Attribute Type		Transformation	Comment
Categorical (Qualitative)	Nominal	Any one-to-one mapping, e.g., a permutation of values	If all employee ID numbers are reassigned, it will not make any difference.
	Ordinal	An order-preserving change of values, i.e., $new_value = f(old_value)$, where f is a monotonic function.	An attribute encompassing the notion of good, better, best can be represented equally well by the values $\{1, 2, 3\}$ or by $\{0.5, 1, 10\}$.
Numeric (Quantitative)	Interval	$new_value = a * old_value + b$, a and b constants.	The Fahrenheit and Celsius temperature scales differ in the location of their zero value and the size of a degree (unit).
	Ratio	$new_value = a * old_value$	Length can be measured in meters or feet.

Describing Attributes by the Number of Values

Discrete :- A **discrete** attribute has a finite or countably infinite set of values. Such attributes can be **categorical**, such as zip codes or ID numbers, or numeric, such as counts. **Discrete** attributes are often represented using integer variables. **Binary** attributes are a special case of discrete attributes and assume only two values, e.g., true/false, yes/no, male/female, or 0/1. **Binary** attributes

are often represented as **Boolean** variables, or as **integer** variables that only take the values 0 or 1.

Continuous :- A **continuous** attribute is one whose values are real numbers. **Examples** include attributes such as temperature, height, or weight. Continuous attributes are typically represented as floating-point variables. Practically, real values can only be measured and represented with limited precision.

Typically, **nominal** and **ordinal** attributes are **binary** or **discrete**, while **interval** and **ratio** attributes are **continuous**. However, **count** attributes, which are **discrete**, are also **ratio** attributes.

Asymmetric Attributes

For **asymmetric** attributes, only presence a non-zero attribute value is regarded as important.

Asymmetric binary attributes :- Binary attributes where only non-zero values are important are called **asymmetric binary attributes**.

Asymmetric discrete or **continuous attributes** :- If the number of credits associated with each course is recorded, then the resulting data set will consist of **asymmetric discrete** or **continuous attributes**.

Types of Data Sets

For convenience, we have grouped the types of data sets into three groups: **record data, graph-based data,** and **ordered data**. These categories do not cover all possibilities and other groupings are certainly possible.

General Characteristics of Data Sets

Dimensionality :- The **dimensionality** of a data set is the number of attributes that the objects in the data set possess. Data with a small number of dimensions tends to be qualitatively different than moderate or high-dimensional data. Indeed, the difficulties associated with analyzing high-dimensional data are sometimes referred to as the **curse of dimensionality**. Because of this, an important motivation in preprocessing the data is **dimensionality reduction**.

Sparsity :- For some data sets, such as those with asymmetric features, most attributes of an object have values of 0; in many cases, fewer than 1% of the entries are non-zero. In practical terms, **sparsity** is an advantage because usually only the non-zero values need to be stored and manipulated. This results in significant savings with respect to computation time and storage. Furthermore, some data mining algorithms work well only for sparse data.

DEEPAK RAJPUT

Resolution :- It is frequently possible to obtain data at different levels of resolution, and often the properties of the data are different at different resolutions. For instance, the surface of the Earth seems very uneven at a resolution of a few meters, but is relatively smooth at a resolution of tens of kilometers. The patterns in the data also depend on the level of resolution. If the resolution is too fine, a pattern may not be visible or may be buried in noise; if the resolution is too coarse, the pattern may disappear. For example, variations in atmospheric pressure on a scale of hours reflect the movement of storms and other weather systems. On a scale of months, such phenomena are not detectable.

Record Data

For the most basic form of record data, there is no explicit relationship among records or data fields, and every record (object) has the same set of attributes. Record data is usually stored either in flat files or in relational databases. Relational databases are certainly more than a collection of records, but data mining often does not use any of the additional information available in a relational database.

Transaction or Market Basket Data

Transaction data is a special type of record data, where each record (transaction) involves a set of items.

This type of data is called market basket data because the items in each record are the products in a person's "market basket." Transaction data is a collection of sets of items, but it can be viewed as a set of records whose fields are asymmetric attributes.

The Data Matrix

If the data objects in a collection of data all have the same fixed set of numeric attributes, then the data objects can be thought of as points (vectors) in a multidimensional space, where each dimension represents a distinct attribute describing the object. A set of such data objects can be interpreted as an m by n matrix, where there are m rows, one for each object, and n columns, one for each attribute. (A representation that has data objects as columns and attributes as rows is also fine.) This matrix is called a **data matrix** or a **pattern matrix**. A data matrix is a variation of record data, but because it consists of numeric attributes, standard matrix operation can be applied to transform and manipulate the data.

DEEPAK RAJPUT

Tid	Refund	Marital Status	Taxable Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

(a) Record data.

TID	ITEMS
1	Bread, Soda, Milk
2	Beer, Bread
3	Beer, Soda, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Soda, Diaper, Milk

(b) Transaction data.

Projection of x Load	Projection of y Load	Distance	Load	Thickness
10.23	5.27	15.22	27	1.2
12.65	6.25	16.22	22	1.1
13.54	7.23	17.34	23	1.2
14.27	8.43	18.45	25	0.9

(c) Data matrix.

	team	coach	play	ball	score	game	win	lost	timeout	season
Document 1	3	0	5	0	2	6	0	2	0	2
Document 2	0	7	0	2	1	0	0	3	0	0
Document 3	0	1	0	0	1	2	2	0	3	0

(d) Document-term matrix.

Figure 2.2. Different variations of record data.

The Sparse Data Matrix

A sparse data matrix is a special case of a data matrix in which the attributes are of the same type and are asymmetric; i.e., only non-zero values are important.

In particular, if the order of the terms (words) in a document is ignored, then a document can be represented as a term vector, where each term is a component (attribute) of the vector and the value of each component is the number of times the corresponding term occurs in the document. This representation of a collection of documents is often called a **document-term matrix.**

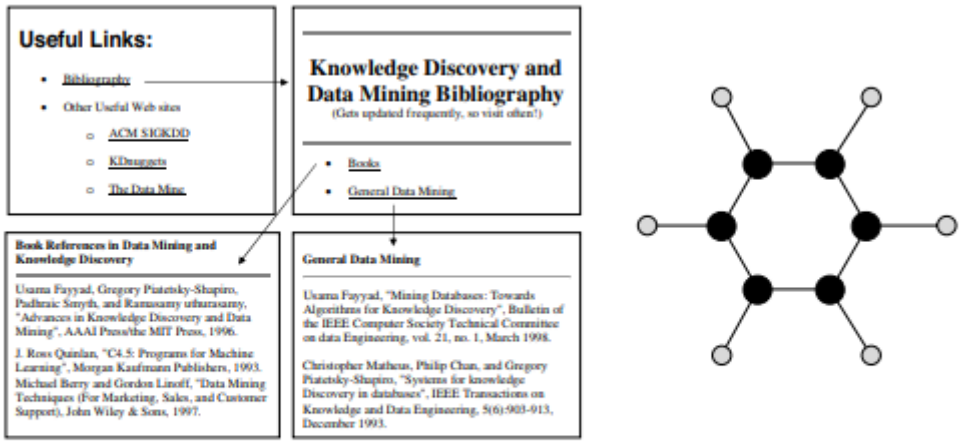
Graph-Based Data

A graph can sometimes be a convenient and powerful representation for data. We consider two specific cases:

(1) the graph captures relationships among data objects and

(2) the data objects themselves are represented as graphs.

DEEPAK



(a) Linked Web pages. (b) Benzene molecule.

Figure 2.3. Different variations of graph data.

Data with Relationships among Objects

The relationships among objects frequently convey important information. In such cases, the data is often represented as a graph. In particular, the data objects are mapped to nodes of the graph, while the relationships among objects are captured by the links between objects and link properties, such as direction and weight.

Data with Objects That Are Graphs

If objects have structure, that is, the objects contain sub-objects that have relationships, then such objects are frequently represented as graphs. For example, the structure of chemical compounds can be represented by a graph, where the nodes are atoms and the links between nodes are chemical bonds.

Ordered Data

Sequential Data

Sequential data, also referred to as **temporal data**, can be thought of as an extension of record data, where each record has a time associated with it. Consider a retail transaction data set that also stores the time at which the transaction took place. This time information makes it possible to find patterns such as "candy sales peak before Halloween." A time can also be associated with each attribute.

DEEPAK RAJPUT

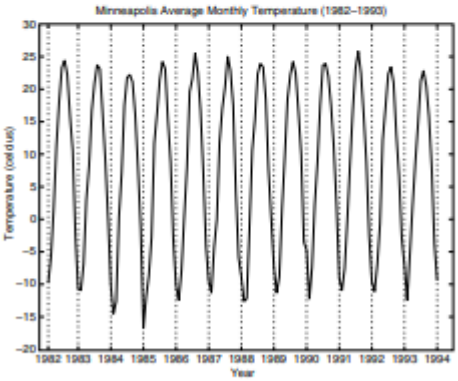
Time	Customer	Items Purchased
t1	C1	A, B
t2	C3	A, C
t2	C1	C, D
t3	C2	A, D
t4	C2	E
t5	C1	A, E

Customer	Time and Items Purchased
C1	(t1: A,B) (t2:C,D) (t5:A,E)
C2	(t3: A, D) (t4: E)
C3	(t2: A, C)

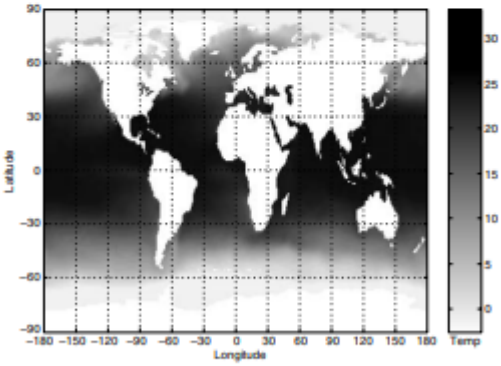
(a) Sequential transaction data.

```
GGTTCCGCCTTCAGCCCCGCGCC
CGCAGGGCCCGCCCCGCGCCGTC
GAGAAGGGCCCGCCTGGCGGGCG
GGGGGAGGCGGGGCCGCCCGAGC
CCAACCGAGTCCGACCAGGTGCC
CCCTCTGCTCGGCCTAGACCTGA
GCTCATTAGGCGGCAGCGGACAG
GCCAAGTAGAACACGCGAAGCGC
TGGGCTGCCTGCTGCGACCAGGG
```

(b) Genomic sequence data.



(c) Temperature time series.



(d) Spatial temperature data.

Figure 2.4. Different variations of ordered data.

Sequence Data

Sequence data consists of a data set that is a sequence of individual entities, such as a sequence of words or letters. It is quite similar to sequential data, except that there are no time stamps; instead, there are positions in an ordered sequence.

Time Series Data

Time series data is a special type of sequential data in which each record is a **time series**, i.e., a series of measurements taken over time.

When working with temporal data, it is important to consider **temporal autocorrelation**; i.e., if two measurements are close in time, then the values of those measurements are often very similar.

Spatial Data

Some objects have spatial attributes, such as positions or areas, as well as other types of attributes. An example of spatial data is weather data (precipitation,

temperature, pressure) that is collected for a variety of geographical locations. An important aspect of spatial data is **spatial autocorrelation**; i.e., objects that are physically close tend to be similar in other ways as well. Thus, two points on the Earth that are close to each other usually have similar values for temperature and rainfall.

Handling Non-Record Data

Record-oriented techniques can be applied to non-record data by extracting features from data objects and using these features to create a record corresponding to each object.

Given a set of common substructures, each compound can be represented as a record with binary attributes that indicate whether a compound contains a specific substructure. Such a representation is actually a transaction data set, where the transactions are the compounds and the items are the substructures.

Consider spatio-temporal data consisting of a time series from each point on a spatial grid. This data is often stored in a data matrix, where each row represents a location and each column represents a particular point in time. However, such a representation does not explicitly capture the time relationships that are present among attributes and the spatial relationships that exist among objects.

Data Quality

Data mining applications are often applied to data that was collected for another purpose, or for future, but unspecified applications. For that reason, data mining cannot usually take advantage of the significant benefits of "addressing quality issues at the source."

Because preventing data quality problems is typically not an option, data mining focuses on

(1) the detection and correction of data quality problems and

(2) the use of algorithms that can tolerate poor data quality.

The first step, detection and correction, is often called **data cleaning**.

Measurement and Data Collection Issues

It is unrealistic to expect that data will be perfect. There may be problems due to human error, limitations of measuring devices, or flaws in the data collection

DEEPAK RAJPUT

process. Values or even entire data objects may be missing. In other cases, there may be spurious or duplicate objects; i.e., multiple data objects that all correspond to a single "real" object.

For example, there might be two different records for a person who has recently lived at two different addresses. Even if all the data is present and "looks fine," there may be inconsistencies—a person has a height of 2 meters, but weighs only 2 kilograms.

Measurement and Data Collection Errors

The term **measurement error** refers to any problem resulting from the measurement process. A common problem is that the value recorded differs from the true value to some extent. For continuous attributes, the numerical difference of the measured and true value is called the **error**. The term **data collection error** refers to errors such as omitting data objects or attribute values, or inappropriately including a data object.

For example, a study of animals of a certain species might include animals of a related species that are similar in appearance to the species of interest. Both measurement errors and data collection errors can be either systematic or random.

Noise and Artifacts

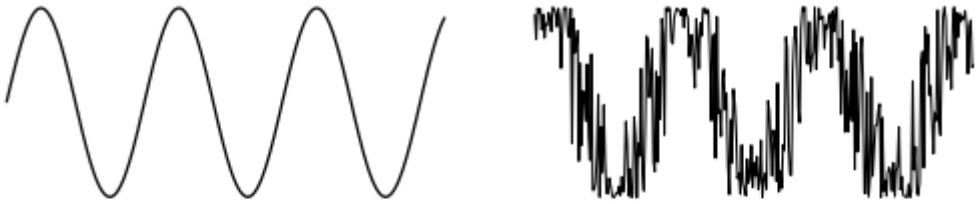
Noise is the random component of a measurement error. It may involve the distortion of a value or the addition of spurious objects.

The term noise is often used in connection with data that has a spatial or temporal component.

In such cases, techniques from signal or image processing can frequently be used to reduce noise and thus, help to discover patterns (signals) that might be "lost in the noise." Nonetheless, the elimination of noise is frequently difficult, and much work in data mining focuses on devising robust algorithms that produce acceptable results even when noise is present.

Data errors may be the result of a more deterministic phenomenon, such as a streak in the same place on a set of photographs. Such deterministic distortions of the data are often referred to as **artifacts**.

DEEPAK RAJPUT



(a) Time series. (b) Time series with noise.

Figure 2.5. Noise in a time series context.

(a) Three groups of points. (b) With noise points (+) added.

Figure 2.6. Noise in a spatial context.

Precision, Bias, and Accuracy

Precision: The closeness of repeated measurements (of the same quantity) to one another.

Bias: A systematic variation of measurements from the quantity being measured.

Accuracy: The closeness of measurements to the true value of the quantity being measured.

Outliers

Outliers are either (1) data objects that, in some sense, have characteristics that are different from most of the other data objects in the data set, or (2) values of an attribute that are unusual with respect to the typical values for that attribute.

Alternatively, we can speak of **anomalous** objects or values.

DEEPAK

Missing Values

It is not unusual for an object to be missing one or more attribute values. In some cases, the information was not collected; e.g., some people decline to give their age or weight.

There are several strategies (and variations on these strategies) for dealing with missing data, each of which may be appropriate in certain circumstances. These strategies are listed next, along with an indication of their advantages and disadvantages.

Eliminate Data Objects or Attributes

A simple and effective strategy is to eliminate objects with missing values. However, even a partially specified data object contains some information, and if many objects have missing values, then a reliable analysis can be difficult or impossible.

Estimate Missing Values

Sometimes missing data can be reliably estimated. For example, consider a time series that changes in a reasonably smooth fashion, but has a few, widely scattered missing values. In such cases, the missing values can be estimated (interpolated) by using the remaining values.

Ignore the Missing Value during Analysis

Many data mining approaches can be modified to ignore missing values. For example, suppose that objects are being clustered and the similarity between pairs of data objects needs to be calculated. If one or both objects of a pair have missing values for some attributes, then the similarity can be calculated by using only the attributes that do not have missing values.

Inconsistent Values

Data can contain **inconsistent** values. Consider an address field, where both a zip code and city are listed, but the specified zip code area is not contained in that city. It may be that the individual entering this information transposed two digits, or perhaps a digit was misread when the information was scanned from a handwritten form. Regardless of the cause of the inconsistent values, it is important to detect and, if possible, correct such problems.

Duplicate Data

A data set may include data objects that are **duplicates**, or almost **duplicates**, of one another. Many people receive duplicate mailings because they appear in a

DEEPAK RAJPUT

database multiple times under slightly different names. To detect and eliminate such duplicates, two main issues must be addressed. First, if there are two objects that actually represent a single object, then the values of corresponding attributes may differ, and these inconsistent values must be resolved. Second, care needs to be taken to avoid accidentally combining data objects that are similar, but not duplicates, such as two distinct people with identical names. The term deduplication is often used to refer to the process of dealing with these issues.

Issues Related to Applications

Data quality issues can also be considered from an application viewpoint as expressed by the statement "**data is of high quality if it is suitable for its intended use**."

Timeliness

Some data starts to age as soon as it has been collected. In particular, if the data provides a snapshot of some ongoing phenomenon or process, such as the purchasing behavior of customers or Web browsing patterns, then this snapshot represents reality for only a limited time. If the data is out of date, then so are the models and patterns that are based on it.

Relevance

The available data must contain the information necessary for the application. Consider the task of building a model that predicts the accident rate for drivers. If information about the age and gender of the driver is omitted, then it is likely that the model will have limited accuracy unless this information is indirectly available through other attributes.

A common problem is **sampling bias**, which occurs when a sample does not contain different types of objects in proportion to their actual occurrence in the population.

Knowledge about the Data

Ideally, data sets are accompanied by documentation that describes different aspects of the data; the quality of this documentation can either aid or hinder the subsequent analysis.

DEEPAK RAJPUT

Data Preprocessing

Data preprocessing is a broad area and consists of a number of different strategies and techniques that are interrelated in complex ways.

Specifically, we will discuss the following topics:

- Aggregation
- Sampling
- Dimensionality reduction
- Feature subset selection
- Feature creation
- Discretization and binarization
- Variable transformation

Aggregation

Sometimes "less is more" and this is the case with **aggregation**, the combining of two or more objects into a single object.

Quantitative attributes, such as price, are typically aggregated by taking a sum or an average. A qualitative attribute, such as item, can either be omitted or summarized as the set of all the items that were sold at that location.

Aggregation is the process of eliminating attributes, such as the type of item, or reducing the number of values for a particular attribute; e.g., reducing the possible values for date from 365 days to 12 months. This type of aggregation is commonly used in **Online Analytical Processing (OLAP)**.

There are several motivations for aggregation. **First**, the smaller data sets resulting from data reduction require less memory and processing time, and hence, aggregation may permit the use of more expensive data mining algorithms. **Second**, aggregation can act as a change of scope or scale by providing a high-level view of the data instead of a low-level view.

A **disadvantage** of aggregation is the potential loss of interesting details. In the store example aggregating over months loses information about which day of the week has the highest sales.

Sampling

Sampling is a commonly used approach for selecting a subset of the data objects to be analyzed.

DEEPAK RAJPUT

The key principle for effective sampling is the following: Using a sample will work almost as well as using the entire data set if the sample is **representative**. In turn, a sample is representative if it has approximately the same property (of interest) as the original set of data.

Sampling Approaches

The simplest type of sampling is **simple random sampling**. For this type of sampling, there is an equal probability of selecting any particular item. There are two variations on random sampling (and other sampling techniques as well):

(1)**Sampling without replacement**—as each item is selected, it is removed from the set of all objects that together constitute the population, and

(2)**Sampling with replacement**—objects are not removed from the population as they are selected for the sample. In sampling with replacement, the same object can be picked more than once.

The samples produced by the two methods are not much different when samples are relatively small compared to the data set size, but sampling with replacement is simpler to analyze since the probability of selecting any object remains constant during the sampling process.

Stratified sampling, which starts with prespecified groups of objects, is such an approach. In the simplest version, equal numbers of objects are drawn from each group even though the groups are of different sizes. In another variation, the number of objects drawn from each group is proportional to the size of that group.

Progressive Sampling

The proper sample size can be difficult to determine, so **adaptive** or **progressive sampling** schemes are sometimes used. These approaches start with a small sample, and then increase the sample size until a sample of sufficient size has been obtained. While this technique eliminates the need to determine the correct sample size initially, it requires that there be a way to evaluate the sample to judge if it is large enough.

Dimensionality Reduction

The term **dimensionality reduction** is often reserved for those techniques that reduce the dimensionality of a data set by creating new attributes that are a combination of the old attributes.

DEEPAK RAJPUT

A key **benefit** is that many data mining algorithms work better if the dimensionality—the number of attributes in the data—is lower.

A reduction of dimensionality can lead to a more understandable model because the model may involve fewer attributes. Also, dimensionality reduction may allow the data to be more easily visualized.

The Curse of Dimensionality

The curse of dimensionality refers to the phenomenon that many types of data analysis become significantly harder as the dimensionality of the data increases. Specifically, as dimensionality increases, the data becomes increasingly sparse in the space that it occupies.

For classification, this can mean that there are not enough data objects to allow the creation of a model that reliably assigns a class to all possible objects.

For clustering, the definitions of density and the distance between points, which are critical for clustering, become less meaningful.

Linear Algebra Techniques for Dimensionality Reduction

Principal Components Analysis (PCA) is a linear algebra technique for continuous attributes that finds new attributes (principal components) that

(1) are linear combinations of the original attributes,
(2) are orthogonal (perpendicular) to each other, and
(3) capture the maximum amount of variation in the data.

For example, the first two principal components capture as much of the variation in the data as is possible with two orthogonal attributes that are linear combinations of the original attributes.

Singular Value Decomposition (SVD) is a linear algebra technique that is related to PCA and is also commonly used for dimensionality reduction.

Feature Subset Selection

Redundant features duplicate much or all of the information contained in one or more other attributes. For example, the purchase price of a product and the amount of sales tax paid contain much of the same information.

Irrelevant features contain almost no useful information for the data mining task at hand. For instance, students' ID numbers are irrelevant to the task of predicting students' grade point averages.

DEEPAK RAJPUT

There are three standard approaches to feature selection: embedded, filter, and wrapper.

Embedded approaches Feature selection occurs naturally as part of the data mining algorithm. Specifically, during the operation of the data mining algorithm, the algorithm itself decides which attributes to use and which to ignore.

Filter approaches Features are selected before the data mining algorithm is run, using some approach that is independent of the data mining task. For example, we might select sets of attributes whose pairwise correlation is as low as possible.

Wrapper approaches These methods use the target data mining algorithm as a black box to find the best subset of attributes, in a way similar to that of the ideal algorithm described above, but typically without enumerating all possible subsets.

An Architecture for Feature Subset Selection

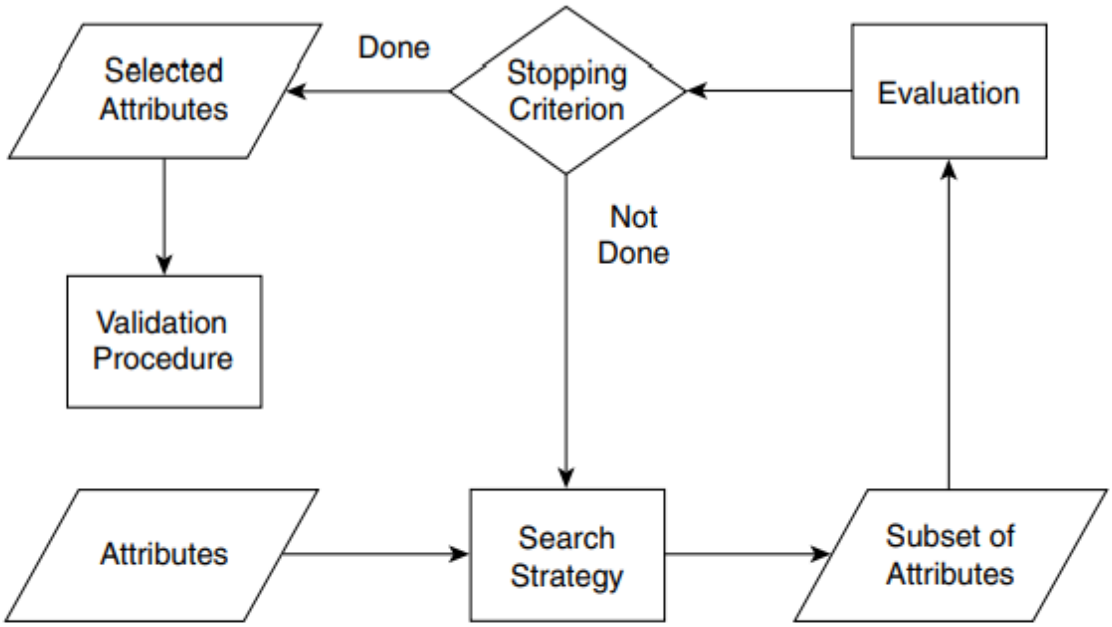


Figure 2.11. Flowchart of a feature subset selection process.

Feature Weighting

Feature weighting is an alternative to keeping or eliminating features. More important features are assigned a higher weight, while less important features are given a lower weight.

These weights are sometimes assigned based on domain knowledge about the relative importance of features. Alternatively, they may be determined automatically.

Feature Creation

Three related methodologies for creating new attributes are described next: feature extraction, mapping the data to a new space, and feature construction.

Feature Extraction

The creation of a new set of features from the original raw data is known as **feature extraction**. Consider a set of photographs, where each photograph is to be classified according to whether or not it contains a human face. The raw data is a set of pixels, and as such, is not suitable for many types of classification algorithms. However, if the data is processed to provide higherlevel features, such as the presence or absence of certain types of edges and areas that are highly correlated with the presence of human faces, then a much broader set of classification techniques can be applied to this problem.

Mapping the Data to a New Space

A totally different view of the data can reveal important and interesting features. Consider, for example, time series data, which often contains periodic patterns. If there is only a single periodic pattern and not much noise, then the pattern is easily detected. If, on the other hand, there are a number of periodic patterns and a significant amount of noise is present, then these patterns are hard to detect. Such patterns can, nonetheless, often be detected by applying a **Fourier transform** to the time series in order to change to a representation in which frequency information is explicit.

Many other sorts of transformations are also possible. Besides the Fourier transform, the **wavelet transform** has also proven very useful for time series and other types of data.

Feature Construction

Sometimes the features in the original data sets have the necessary information, but it is not in a form suitable for the data mining algorithm. In this situation, one or more new features constructed out of the original features can be more useful than the original features.

DEEPAK RAJPUT

Discretization and Binarization

Algorithms that find association patterns require that the data be in the form of binary attributes. Thus, it is often necessary to transform a continuous attribute into a categorical attribute (**discretization**), and both continuous and discrete attributes may need to be transformed into one or more binary attributes (**binarization**).

Table 2.5. Conversion of a categorical attribute to three binary attributes.

Categorical Value	Integer Value	x_1	x_2	x_3
awful	0	0	0	0
poor	1	0	0	1
OK	2	0	1	0
good	3	0	1	1
great	4	1	0	0

Table 2.6. Conversion of a categorical attribute to five asymmetric binary attributes.

Categorical Value	Integer Value	x_1	x_2	x_3	x_4	x_5
awful	0	1	0	0	0	0
poor	1	0	1	0	0	0
OK	2	0	0	1	0	0
good	3	0	0	0	1	0
great	4	0	0	0	0	1

Discretization of Continuous Attributes

Entropy

First, it is necessary to define entropy.

Let k be the number of different class labels,

mi be the number of values in the i th interval of a partition, and

mij be the number of values of class j in interval i.

Then the entropy ei of the i th interval is given by the equation

$$e_i = \sum_{i=1}^{k} p_{ij} \log_2 p_{ij},$$

where pij = mij/mi is the probability (fraction of values) of class j in the i th interval.

The total entropy, e, of the partition is the weighted average of the individual interval entropies, i.e.,

$$e = \sum_{i=1}^{n} w_i e_i,$$

where m is the number of values, wi = mi/m is the fraction of values in the i th interval, and n is the number of intervals.

Variable Transformation

A **variable transformation** refers to a transformation that is applied to all the values of a variable.

Simple Functions

For this type of variable transformation, a simple mathematical function is applied to each value individually. If x is a variable, then examples of such transformations include xk, log x, ex, √x, 1/x, sin x, or |x|. In statistics, variable transformations, especially sqrt, log, and 1/x, are often used to transform data that does not have a Gaussian (normal) distribution into data that does.

Normalization or Standardization

Another common type of variable transformation is the **standardization** or **normalization** of a variable.

The goal of standardization or normalization is to make an entire set of values have a particular property. A traditional example is that of "standardizing a variable" in statistics. If $\bar{x}$ is the mean (average) of the attribute values and S_x is their standard deviation, then the transformation $x' = (x - \bar{x})/S_x$ creates a new variable that has a mean of 0 and a standard deviation of 1.

The mean and standard deviation are strongly affected by **outliers**, so the above transformation is often modified. First, the mean is replaced by the **median**, i.e., the middle value. Second, the standard deviation is replaced by the **absolute standard deviation**. Specifically, if x is a variable, then the absolute standard deviation of x is given by

$$\sigma_A = \sum_{i=1}^{m} |x_i - \mu|$$

DEEPAK RAJPUT

where xi is the ith value of the variable, m is the number of objects, and μ is either the mean or median.

Measures of Similarity and Dissimilarity

Similarity The similarity between two objects is a numerical measure of the degree to which the two objects are alike. Consequently, similarities are higher for pairs of objects that are more alike. Similarities are usually non-negative and are often between 0 (no similarity) and 1 (complete similarity).

Dissimilarity The dissimilarity between two objects is a numerical measure of the degree to which the two objects are different. Dissimilarities are lower for more similar pairs of objects. Frequently, the term distance is used as a synonym for dissimilarity, although, as we shall see, distance is often used to refer to a special class of dissimilarities.

Dissimilarities sometimes fall in the interval [0, 1], but it is also common for them to range from 0 to ∞.

Dissimilarities between Data Objects

Distances

The **Euclidean distance**, d, between two points, x and y, in one-, two-, three-, or higher dimensional space, is given by the following familiar formula:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^{n} (x_k - y_k)^2},$$

where n is the number of dimensions and xk and yk are, respectively, the kth attributes (components) of x and y.

The Euclidean distance measure is generalized by the **Minkowski distance** metric.

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r},$$

where r is a parameter. The following are the three most common examples of Minkowski distances.

DEEPAK RAJPUT

• r = 1. City block (Manhattan, taxicab, L1 norm) distance. A common example is the Hamming distance, which is the number of bits that are different between two objects that have only binary attributes, i.e., between two binary vectors.

• r = 2. Euclidean distance (L2 norm).

• r = ∞. Supremum (Lmax or L∞ norm) distance. This is the maximum difference between any attribute of the objects.

$$d(\mathbf{x}, \mathbf{y}) = \lim_{r \to \infty} \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r}.$$

Distances, such as the Euclidean distance, have some well-known properties. If d(x, y) is the distance between two points, x and y, then the following properties hold.

1. **Positivity**

(a) d(x, x) ≥ 0 for all x and y,

(b) d(x, y) = 0 only if x = y.

2. **Symmetry**

d(x, y) = d(y, x) for all x and y.

3. **Triangle Inequality**

d(x, z) ≤ d(x, y) + d(y, z) for all points x, y, and z

Measures that satisfy all three properties are known as **metrics**.

DEEPAK RAJPUT

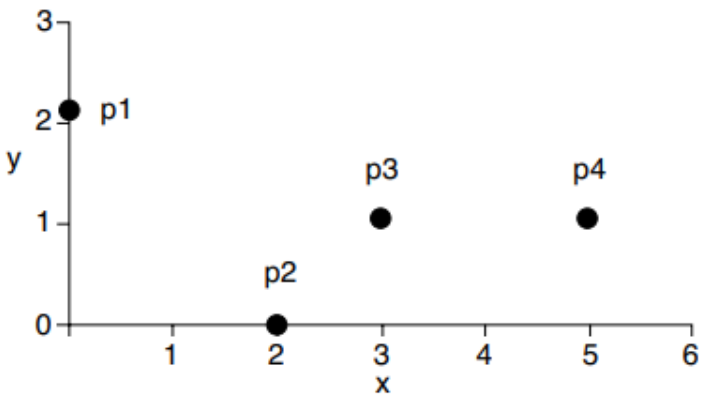


Figure 2.15. Four two-dimensional points.

Table 2.8. x and y coordinates of four points.

point	x coordinate	y coordinate
p1	0	2
p2	2	0
p3	3	1
p4	5	1

Table 2.9. Euclidean distance matrix for Table 2.8.

	p1	p2	p3	p4
p1	0.0	2.8	3.2	5.1
p2	2.8	0.0	1.4	3.2
p3	3.2	1.4	0.0	2.0
p4	5.1	3.2	2.0	0.0

Table 2.10. L_1 distance matrix for Table 2.8.

L_1	p1	p2	p3	p4
p1	0.0	4.0	4.0	6.0
p2	4.0	0.0	2.0	4.0
p3	4.0	2.0	0.0	2.0
p4	6.0	4.0	2.0	0.0

Table 2.11. L_∞ distance matrix for Table 2.8.

L_∞	p1	p2	p3	p4
p1	0.0	2.0	3.0	5.0
p2	2.0	0.0	1.0	3.0
p3	3.0	1.0	0.0	2.0
p4	5.0	3.0	2.0	0.0

Similarities between Data Objects

For **similarities**, the triangle inequality (or the analogous property) typically does not hold, but symmetry and positivity typically do.

To be explicit, if s(x, y) is the similarity between points x and y, then the typical properties of similarities are the following:

1. s(x, y) = 1 only if x = y. (0 ≤ s ≤ 1)

2. s(x, y) = s(y, x) for all x and y. (Symmetry)

Example 2.16 (A Non-symmetric Similarity Measure). Consider an experiment in which people are asked to classify a small set of characters as they flash on a screen. The **confusion matrix** for this experiment records how often each character is classified as itself, and how often each is classified as another character. For instance, suppose that "0" appeared 200 times and was classified

as a “0” 160 times, but as an “o” 40 times. Likewise, suppose that ‘o’ appeared 200 times and was classified as an “o” 170 times, but as “0” only 30 times. If we take these counts as a measure of the similarity between two characters, then we have a similarity measure, but one that is not symmetric. In such situations, the similarity measure is often made symmetric by setting s’ (x, y) = s’(y, x)=(s(x, y)+s(y, x))/2, where s’ indicates the new similarity measure.

Chapter :- 4

Classification: Basic Concepts, Decision Trees, and Model Evaluation

Classification: Classification is the task of learning a target function f that maps each attribute set x to one of the predefined class labels y.

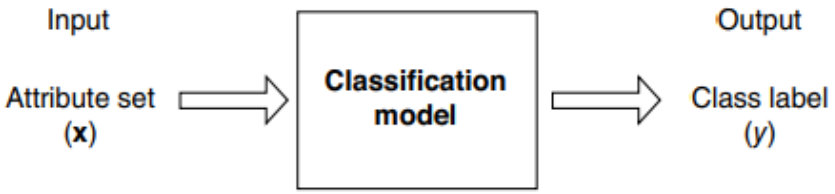


Figure 4.2. Classification as the task of mapping an input attribute set $\mathbf{x}$ into its class label y.

The target function is also known informally as a **classification model.**

Descriptive Modeling A classification model can serve as an explanatory tool to distinguish between objects of different classes.

Predictive Modeling A classification model can also be used to predict the class label of unknown records.

Classification techniques are most suited for predicting or describing data sets with binary or nominal categories. They are less effective for ordinal categories (e.g., to classify a person as a member of high-, medium-, or low-income group) because they do not consider the implicit order among the categories.

General Approach to Solving a Classification Problem

A classification technique (or classifier) is a systematic approach to building classification models from an input data set. Examples include **decision tree classifiers**, **rule-based classifiers**, **neural networks**, **support vector machines**, and **naive Bayes classifiers**. Each technique employs a **learning algorithm** to identify a model that best fits the relationship between the attribute set and class label of the input data. The model generated by a learning algorithm should both fit the input data well and correctly predict the class labels of records it has never seen before. Therefore, a key objective of the learning algorithm is to build models with good generalization capability; i.e., models that accurately predict the class labels of previously unknown records.

First, **a training set** consisting of records whose class labels are known must be provided. The training set is used to build a classification model, which is subsequently applied to the **test set**, which consists of records with unknown class labels.

Evaluation of the performance of a classification model is based on the counts of test records correctly and incorrectly predicted by the model. These counts are tabulated in a table known as a **confusion matrix**.

RAJPUT

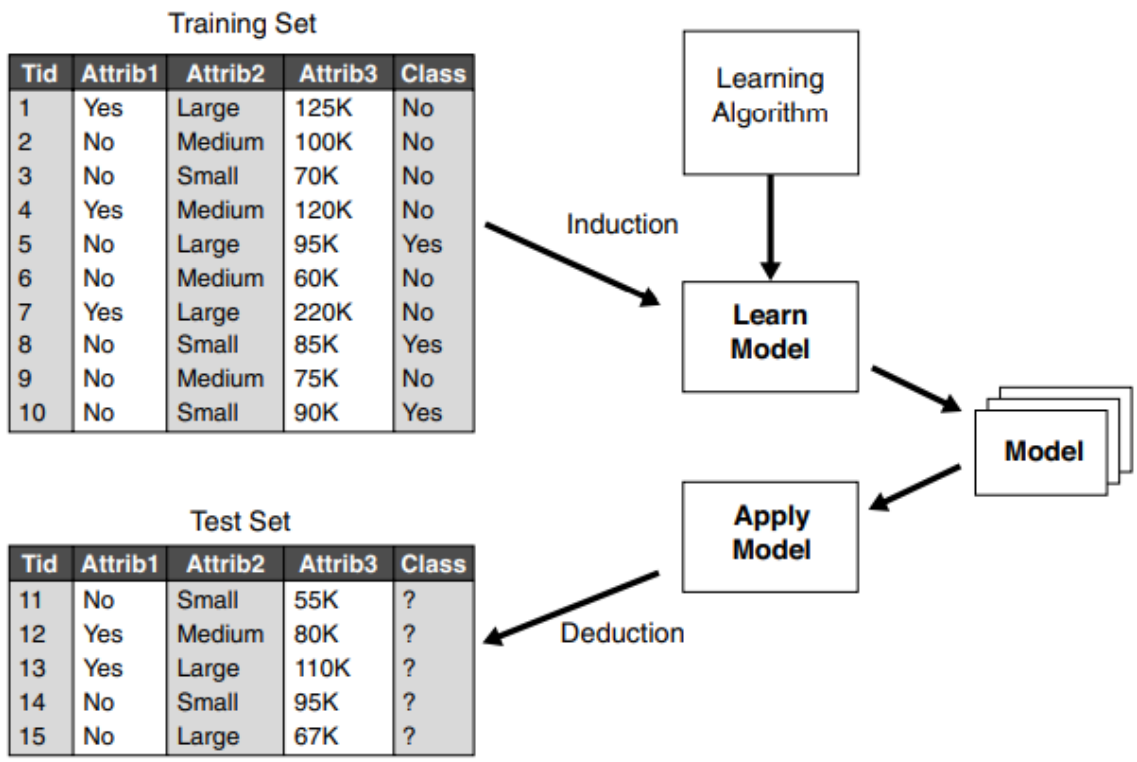


Figure 4.3. General approach for building a classification model.

Table 4.2. Confusion matrix for a 2-class problem.

		Predicted Class	
		$Class = 1$	$Class = 0$
Actual Class	$Class = 1$	f_{11}	f_{10}
	$Class = 0$	f_{01}	f_{00}

Each entry fij in this table denotes the number of records from class i predicted to be of class j. For instance, f01 is the number of records from class 0 incorrectly predicted as class 1. Based on the entries in the confusion matrix, the total number of correct predictions made by the model is (f11 + f00) and the total number of incorrect predictions is (f10 + f01).

This can be done using a performance metric such as accuracy, which is defined as follows:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}} = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}}.$$

Equivalently, the performance of a model can be expressed in terms of its error rate, which is given by the following equation:

$$\text{Error rate} = \frac{\text{Number of wrong predictions}}{\text{Total number of predictions}} = \frac{f_{10} + f_{01}}{f_{11} + f_{10} + f_{01} + f_{00}}.$$

Decision Tree Induction

The tree has three types of nodes:

• A root node that has no incoming edges and zero or more outgoing edges.

• Internal nodes, each of which has exactly one incoming edge and two or more outgoing edges.

• Leaf or terminal nodes, each of which has exactly one incoming edge and no outgoing edges.

In a decision tree, each leaf node is assigned a class label. The nonterminal nodes, which include the root and other internal nodes, contain attribute test conditions to separate records that have different characteristics.

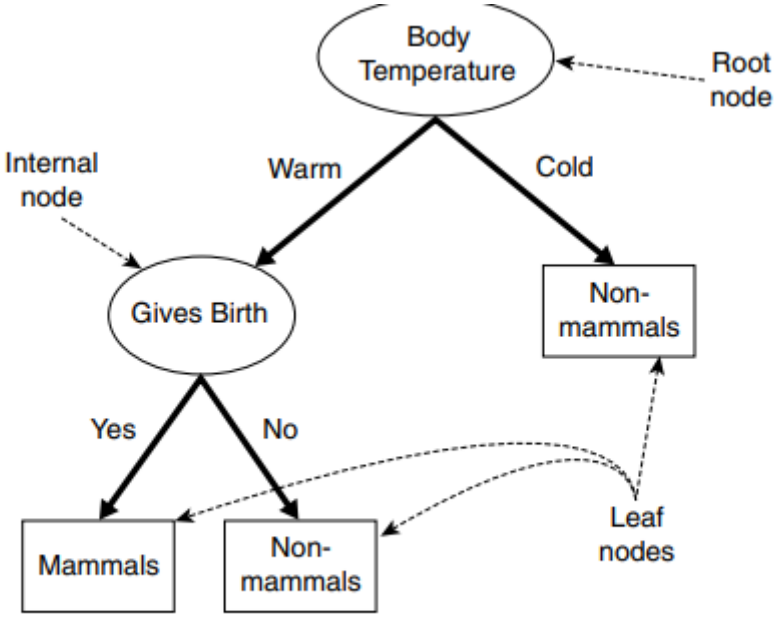


Figure 4.4. A decision tree for the mammal classification problem.

How to Build a Decision Tree

One such algorithm is Hunt's algorithm, which is the basis of many existing decision tree induction algorithms, including ID3, C4.5, and CART.

Hunt's Algorithm

In Hunt's algorithm, a decision tree is grown in a recursive fashion by partitioning the training records into successively purer subsets. Let Dt be the set of training records that are associated with node t and y = {y1, y2,...,yc} be the class labels. The following is a recursive definition of Hunt's algorithm.

Step 1: If all the records in Dt belong to the same class yt, then t is a leaf node labeled as yt.

Step 2: If Dt contains records that belong to more than one class, an attribute test condition is selected to partition the records into smaller subsets. A child node is created for each outcome of the test condition and the records in Dt are distributed to the children based on the outcomes. The algorithm is then recursively applied to each child node.

DEEPAK RAJ

	binary	categorical	continuous	class
Tid	**Home Owner**	**Marital Status**	**Annual Income**	**Defaulted Borrower**
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Figure 4.6. Training set for predicting borrowers who will default on loan payments.

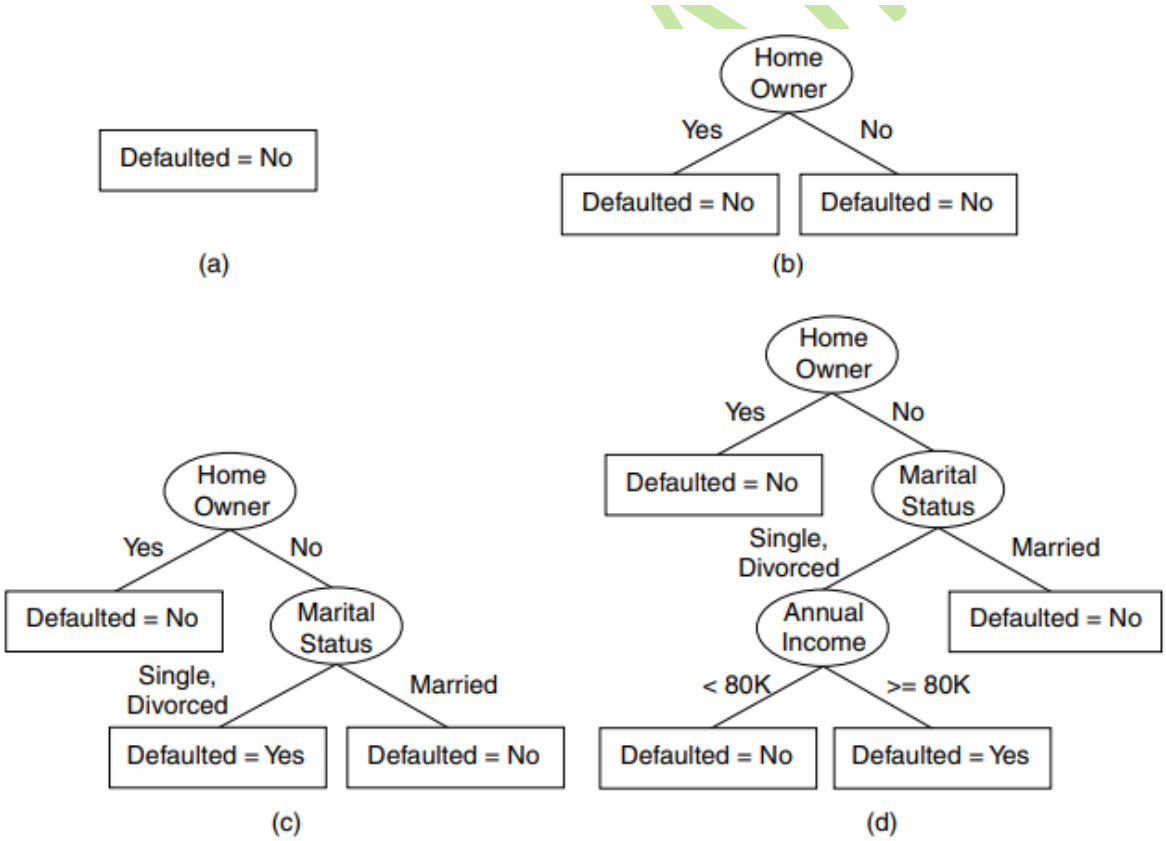


Figure 4.7. Hunt's algorithm for inducing decision trees.

Hunt's algorithm will work if every combination of attribute values is present in the training data and each combination has a unique class label.

These assumptions are too stringent for use in most practical situations. Additional conditions are needed to handle the following cases:

1. It is possible for some of the child nodes created in Step 2 to be empty; i.e., there are no records associated with these nodes. This can happen if none of the training records have the combination of attribute values associated with such nodes. In this case the node is declared a leaf node with the same class label as the majority class of training records associated with its parent node.

2. In Step 2, if all the records associated with Dt have identical attribute values (except for the class label), then it is not possible to split these records any further. In this case, the node is declared a leaf node with the same class label as the majority class of training records associated with this node.

Design Issues of Decision Tree Induction

A learning algorithm for inducing decision trees must address the following two issues.

1. **How should the training records be split?** Each recursive step of the tree-growing process must select an attribute test condition to divide the records into smaller subsets. To implement this step, the algorithm must provide a method for specifying the test condition for different attribute types as well as an objective measure for evaluating the goodness of each test condition.

2. **How should the splitting procedure stop?** A stopping condition is needed to terminate the tree-growing process. A possible strategy is to continue expanding a node until either all the records belong to the same class or all the records have identical attribute values. Although both conditions are sufficient to stop any decision tree induction algorithm, other criteria can be imposed to allow the tree-growing procedure to terminate earlier.

Methods for Expressing Attribute Test Conditions

Decision tree induction algorithms must provide a method for expressing an attribute test condition and its corresponding outcomes for different attribute types.

Binary Attributes The test condition for a binary attribute generates two potential outcomes.

DEEPAK RAJPUT

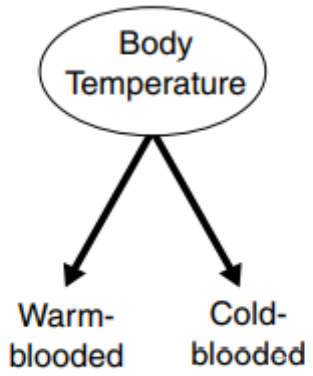


Figure 4.8. Test condition for binary attributes.

Nominal Attributes Since a nominal attribute can have many values, its test condition can be expressed in two ways, as shown in Figure 4.9. For a multiway split (Figure 4.9(a)), the number of outcomes depends on the number of distinct values for the corresponding attribute. For example, if an attribute such as marital status has three distinct values—single, married, or divorced—its test condition will produce a three-way split.

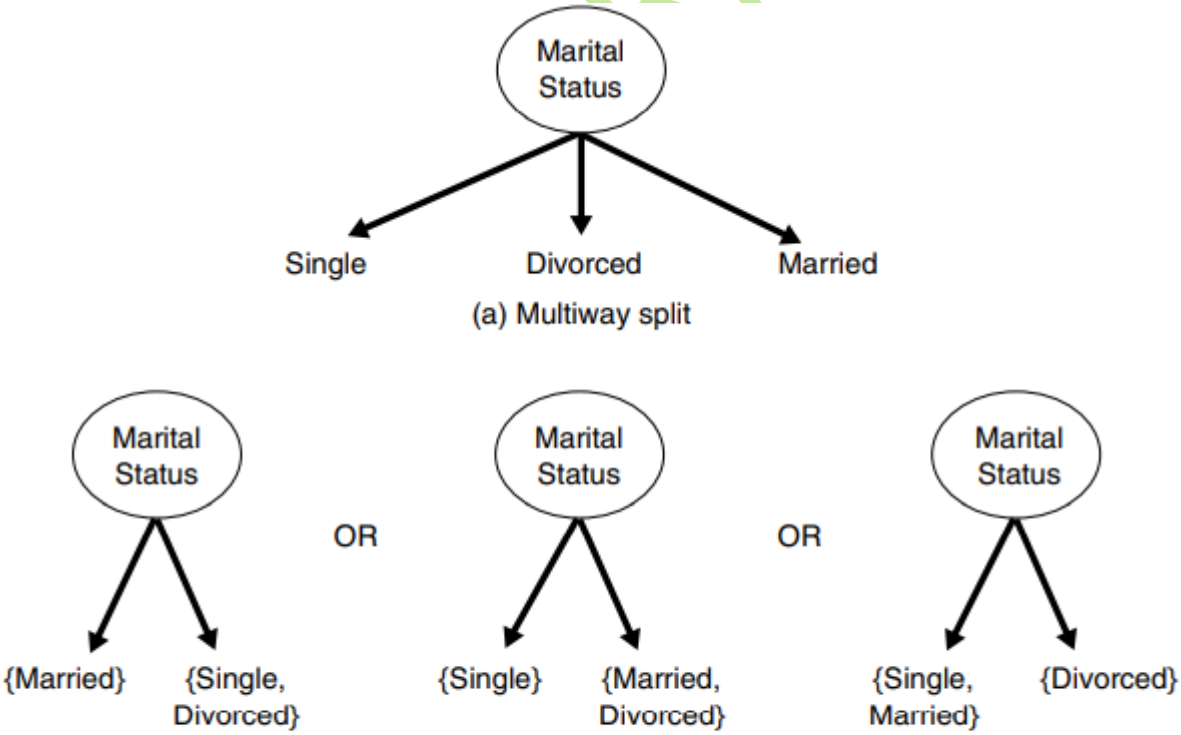


Figure 4.9. Test conditions for nominal attributes.

RAJPUT

Ordinal Attributes Ordinal attributes can also produce binary or multiway splits. Ordinal attribute values can be grouped as long as the grouping does not violate the order property of the attribute values. Figure 4.10 illustrates various ways of splitting training records based on the Shirt Size attribute. The groupings shown in Figures 4.10(a) and (b) preserve the order among the attribute values, whereas the grouping shown in Figure 4.10(c) **violates** this property because it combines the attribute values Small and Large into the same partition while Medium and Extra Large are combined into another partition.

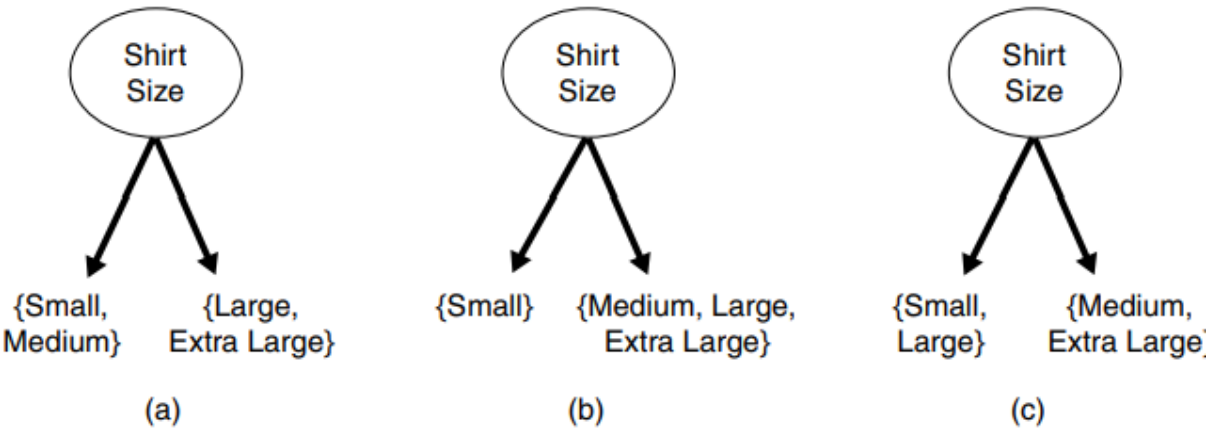


Figure 4.10. Different ways of grouping ordinal attribute values.

Continuous Attributes For continuous attributes, the test condition can be expressed as a comparison test $(A < v)$ or $(A \geq v)$ with binary outcomes, or a range query with outcomes of the form $v_i \leq A < v_{i+1}$, for $i = 1, \ldots, k$. The difference between these approaches is shown in Figure 4.11. For the binary case, the decision tree algorithm must consider all possible split positions v, and it selects the one that produces the best partition. For the multiway split, the algorithm must consider all possible ranges of continuous values.

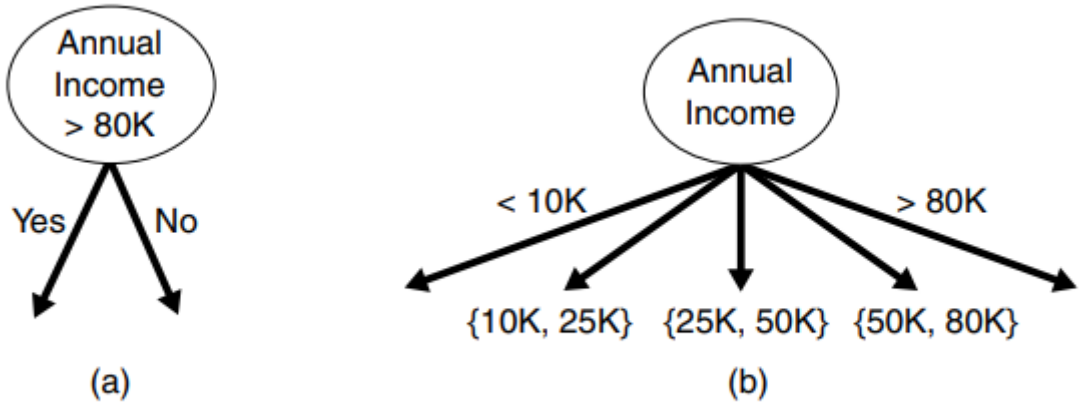


Figure 4.11. Test condition for continuous attributes.

Measures for Selecting the Best Split

There are many measures that can be used to determine the best way to split the records. These measures are defined in terms of the class distribution of the records before and after splitting.

Let p(i|t) denote the fraction of records belonging to class i at a given node t. We sometimes omit the reference to node t and express the fraction as pi. In a two-class problem, the class distribution at any node can be written as (p0, p1), where p1 = 1 − p0. To illustrate, consider the test conditions shown in Figure 4.12. The class distribution before splitting is (0.5, 0.5) because there are an equal number of records from each class.

If we split the data using the Gender attribute, then the class distributions of the child nodes are (0.6, 0.4) and (0.4, 0.6), respectively. Although the classes are no longer evenly distributed, the child nodes still contain records from both classes. Splitting on the second attribute, Car Type, will result in purer partitions.

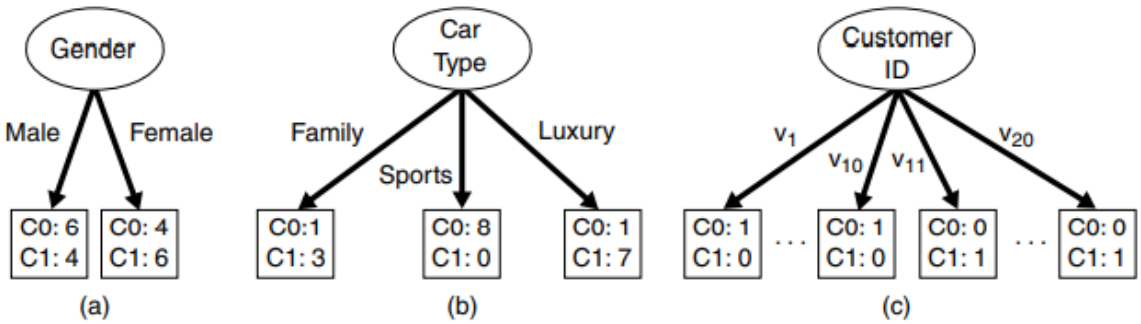


Figure 4.12. Multiway versus binary splits.

For example, a node with class distribu tion (0, 1) has zero impurity, whereas a node with uniform class distribution (0.5, 0.5) has the highest impurity. Examples of impurity measures include:-

$$\begin{aligned}
\text{Entropy}(t) &= -\sum_{i=0}^{c-1} p(i|t)\log_2 p(i|t), \\
\text{Gini}(t) &= 1-\sum_{i=0}^{c-1}[p(i|t)]^2, \\
\text{Classification error}(t) &= 1-\max_i[p(i|t)],
\end{aligned}$$

where c is the number of classes and 0 log2 0 = 0 in entropy calculations.

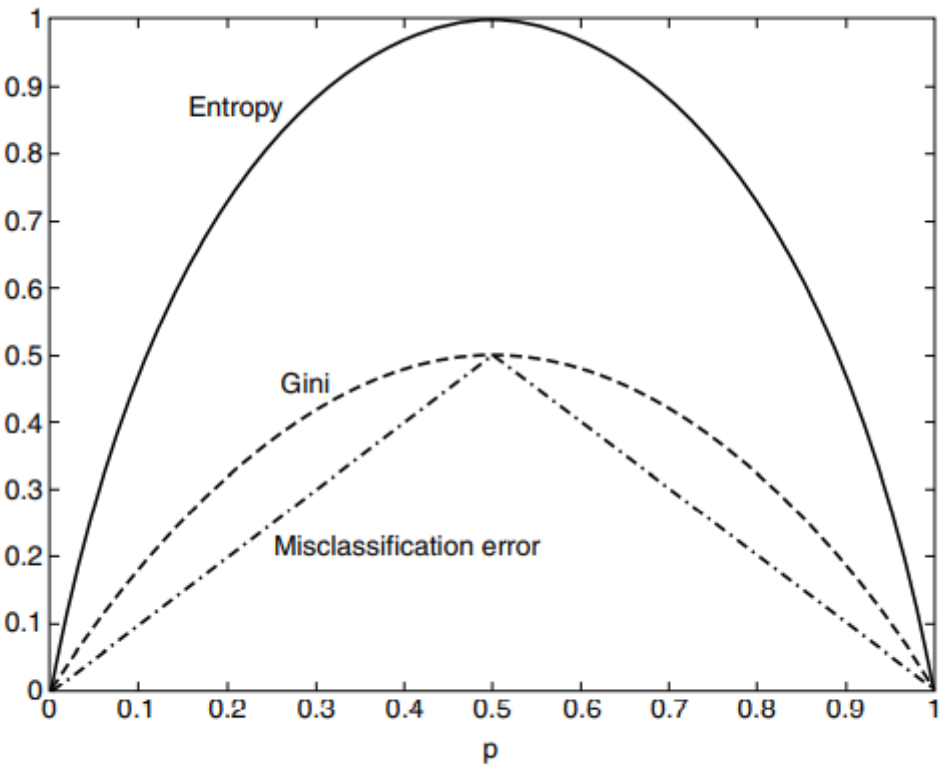


Figure 4.13. Comparison among the impurity measures for binary classification problems.

Figure 4.13 compares the values of the impurity measures for binary classi fication problems. p refers to the fraction of records that belong to one of the two classes. Observe that all three measures attain their maximum value when the class distribution is uniform (i.e., when p = 0.5). The minimum values for the measures are attained when all the records belong to the same class (i.e., when p equals 0 or 1). We next provide several examples of computing the different impurity measures.

Node N_1	Count
Class=0	0
Class=1	6

$$\text{Gini} = 1 - (0/6)^2 - (6/6)^2 = 0$$
$$\text{Entropy} = -(0/6)\log_2(0/6) - (6/6)\log_2(6/6) = 0$$
$$\text{Error} = 1 - \max[0/6, 6/6] = 0$$

Node N_2	Count
Class=0	1
Class=1	5

$$\text{Gini} = 1 - (1/6)^2 - (5/6)^2 = 0.278$$
$$\text{Entropy} = -(1/6)\log_2(1/6) - (5/6)\log_2(5/6) = 0.650$$
$$\text{Error} = 1 - \max[1/6, 5/6] = 0.167$$

Node N_3	Count
Class=0	3
Class=1	3

$$\text{Gini} = 1 - (3/6)^2 - (3/6)^2 = 0.5$$
$$\text{Entropy} = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = 1$$
$$\text{Error} = 1 - \max[3/6, 3/6] = 0.5$$

To determine how well a test condition performs, we need to compare the degree of impurity of the parent node (before splitting) with the degree of impurity of the child nodes (after splitting). The larger their difference, the better the test condition. The gain, Δ, is a criterion that can be used to determine the goodness of a split:

$$\Delta = I(\text{parent}) - \sum_{j=1}^{k} \frac{N(v_j)}{N} I(v_j),$$

where I(·) is the impurity measure of a given node, N is the total number of records at the parent node, k is the number of attribute values, and N(vj) is the number of records associated with the child node, vj . Decision tree induction algorithms often choose a test condition that maximizes the gain Δ. Since I(parent) is the same for all test conditions, maximizing the gain is equivalent to minimizing the weighted average impurity measures of the child nodes. Finally, when entropy is used as the impurity measure, the difference in entropy is known as the **information gain**, Δinfo.

DEEPA PAIDU